

Contexto do projeto — BaF-CoC / Cadeia de Custódia com Blockchain

Anexe este arquivo no início da conversa de implementação. A documentação técnica completa está no `main.tex` / `previa.pdf` (40 páginas): 58 requisitos, 16 entidades com dicionário de dados, arquitetura e sprints.

O que é

Sistema web de gestão da cadeia de custódia de evidências periciais, com registro de integridade em blockchain permissionada. Cobre as dez etapas do art. 158-B do Código de Processo Penal e a Central de Custódia do art. 158-E.

É o trabalho final de curso de Sistemas de Informação na Unifesspa (FACSI), vinculado à dissertação de mestrado de Rodrigo Pereira Gomes Oscar no PPGCF/Unifesspa, que propõe o framework BaF-CoC.

Modalidade do TCC: artigo científico (Art. 8º do Regimento da FACSI — não existe modalidade "produto"; o software é o resultado, o artigo é o entregável à banca).
Aguardando confirmação do coordenador.

Stack

Camada	Tecnologia
Aplicação	ASP.NET Core MVC, .NET 10, Razor
ORM	Entity Framework Core (provider <code>Pomelo.EntityFrameworkCore.MySql</code>)
Banco	MySQL 8
Gateway	Node.js, REST sobre TLS
Wallet	Node.js + SQLite cifrado (scrypt + Ed25519 + Base58)
Ledger	Hyperledger Fabric, consenso Raft, chaincode em JavaScript
Ambiente	Docker Compose, tudo local, sem nuvem
SO do dev	CachyOS (Arch Linux), VSCode

Decisão que mais importa para o código

Implementação a partir da especificação publicada, sem reuso de código.

A arquitetura vem da tese de Leandro Loffi (UFSC, 2025), que descreve em detalhe as funções do chaincode, as rotas do gateway, o esquema de cifragem da wallet e o método `did:legal`. O repositório dele (`leandroloffi/CustodyChain`) **não declara licença**, o que pela Lei 9.610/98 equivale a reserva integral de direitos.

A Lei 9.610/98, art. 8º, I, exclui da proteção autoral "as ideias, procedimentos normativos, sistemas, métodos, projetos ou conceitos matemáticos como tais". Protegido é o código; a especificação publicada não é.

Regra operacional (decisão D-16): não abrir o repositório dele durante a implementação. Implementar lendo a tese. Consultar o código descaracterizaria a independência da obra — e há registro de programa de computador no INPI previsto, onde o campo "Derivação Autorizada" exigiria o contrato de licença que não existe.

Há um pedido de licença pendente por e-mail. Se for concedido antes do início, a decisão é revista.

Arquitetura

```

Navegador (Razor)
  | HTTPS
ASP.NET Core MVC — EF Core — MySQL (estado operacional, 16 entidades)
  |
  | HttpClient (REST/TLS), via interface IServicoLedger
  ▼
Gateway (Node.js) — Wallet (Node.js + SQLite cifrado)
  | Fabric Gateway SDK
  ▼
Hyperledger Fabric — chaincode (DIDs + credenciais verificáveis)
```

Princípio do modelo híbrido: vai para o ledger apenas o que precisa resistir a quem administra o banco — identidade, permissão, hash e prova de custódia. Vestígio, lacre, movimentação e laudo ficam no MySQL. Nenhum dado pessoal ou conteúdo de caso é gravado na rede.

`IServicoLedger` é o único ponto do C# que conhece o gateway. Contrato:

```
csharp
```

```
public interface IServicoLedger
{
    Task<string> GerarDidAsync(TipoAtor tipo);
    Task AtivarDidAsync(string did, string didEmissor, string senhaEmissor);
    Task<DidDocument> ResolverDidAsync(string did);
    Task<string> EmitirCredencialPermissaoAsync(CredencialPermissaoDto dto);
    Task<string> EmitirCredencialCoCAsync(CredencialCoCDto dto);
    Task<ResultadoVerificacao> VerificarCredencialAsync(string credencialJson);
    Task<IReadOnlyList<EstadoRegistro>> HistoricoRegistroAsync(string assetId);
}
```

Uma implementação substituta (`LedgerFake`) permite desenvolver todas as telas antes de o Fabric subir.

Latência conhecida: ~2,3 s por emissão de credencial (medido por Loffi). Por isso toda escrita no ledger é assíncrona: grava no MySQL, enfileira em `REGISTRO_LEDGER` com estado `PENDENTE`, libera o HTTP em menos de 1 s, e um worker consome a fila.

Regras que viram código, não documentação

Regra Efeito no código

RN03 Nenhum registro é excluído, por nenhum perfil. `DeleteBehavior.Restrict` em tudo, e o usuário MySQL da aplicação não recebe `DELETE` nem `DROP`

RN16 Quem cria um registro não pode aprová-lo. `criado_por_id ≠ aprovado_por_id` em MOVIMENTACAO e DESCARTE

RN17 Credencial revogada ou expirada bloqueia a ação, mesmo com usuário autenticado

RN18 Fracionamento gera novos `rotulo_evidencia` sob o mesmo `rotulo_conjunto`; unificação exige conjunto comum

RN19 Recusa reverte ao estado anterior e remove o solicitante — nada fica em limbo

RN21 Divergência de lacre leva a `CustodiaComprometida`, registrado e não reversível em silêncio

Não há coluna de senha no banco. A senha do usuário protege a wallet no dispositivo dele e nunca chega ao servidor. Em SSI não existe "esqueci minha senha" — é limitação conhecida e documentada.

Modelo de dados

16 entidades, dicionário completo no documento técnico (Seção 8):

PERFIL, INTERVENIENTE, PROCESSO, TIPO_VESTIGIO, VESTIGIO, ANEXO, LACRE, MOVIMENTACAO, PERICIA, OPERACAO_AMOSTRA, LAUDO, ARMAZENAMENTO, DESCARTE, CREDENCIAL, REGISTRO_LEDGER, LOG_AUDITORIA

Detalhes que já causaram decisão:

- DATETIME(6), não TIMESTAMP — a faixa do TIMESTAMP acaba em 2038 e o registro digital é perpétuo
- hashes em CHAR(64), comprimento fixo do SHA-256 em hexadecimal
- REGISTRO_LEDGER tem FK polimórfica ((entidade_origem) + (registro_origem_id)), que não pode ser declarada como FK no MySQL. A proteção real é a restrição de unicidade sobre (entidade_origem, registro_origem_id, evento), que garante idempotência
- charset utf8mb4 em todo o esquema

Ordem de construção

Componentes de infraestrutura (Seção 4 do documento técnico):

1. Rede Fabric de teste em contêineres — canal criado, nós respondendo
2. Chaincode: DIDs — gerar, ativar, resolver pela CLI do Fabric
3. Chaincode: credenciais — emitir, atualizar, verificar, revogar, com histórico
4. Wallet — armazenar e recuperar credencial cifrada; senha errada não recupera
5. Gateway — 14 rotas sobre TLS, com erro estruturado
6. IServicoLedger real — trocar o LedgerFake sem alterar controller

Aplicação, em paralelo:

- Projeto .NET, EF Core, migrations das 16 entidades, seed
- Layout base (sidebar retrátil + navbar), LedgerFake
- 11 telas, do login ao verificador independente

Sugestão de ponto de partida: o esqueleto .NET com as migrations. O modelo de dados está 100% especificado, é a stack que o desenvolvedor domina, e destrava todo o trabalho de telas sem depender do Fabric. O ambiente Fabric pode subir em paralelo — e quanto antes ele for testado, antes se descobre se há problema (risco R-04).

As 11 telas

#	Tela
T-01	Login por credencial DID
T-02	Dashboard (varia por perfil)
T-03	Cadastro de vestígio (FAV)
T-04	Recebimento
T-05	Movimentações
T-06	Arquivo
T-07	Destinação final
T-08	Relatórios (PDF e XML)
T-09	Processamento pericial
T-10	Gestão de perfis
T-11	Auditoria e verificador independente

T-11 é a que sustenta a defesa: linha do tempo imutável por vestígio, mais um verificador que confere o hash de um arquivo contra o ledger sem acesso ao sistema.

Perfis

Administrador · Central de Custódia · Agente Coletor · Perito · Órgão externo

Mapeiam para `(did:legal): (admin), (custodian)` (extensão proposta na dissertação do Rodrigo, implementada aqui), `(delegate), (expert)`, e `(judge) / (prosecutor) / (lawyer)`.

Pendências abertas

#	Pendência
P-01	Binários grandes: IPFS privado ou filesystem local
P-02	Assinatura do laudo: Ed25519 da wallet ou ICP-Brasil como evolução
P-03	Autoria no INPI — autor é pessoa física, titular é instituição
P-04	Fabric real desde o início ou <code>LedgerFake</code> primeiro

O que anexar na conversa de implementação

1. Este arquivo
2. `previa.pdf` ou `main.tex` — a documentação técnica completa
3. **Tese do Leandro Loffi** — é a especificação de onde sai o chaincode, o gateway e a wallet. Capítulo 4 é o que importa